EXNO: 1 DATE:	With Weka Data Exploration And Integration

AIM:

To Explore Data and Integrate with WEKA

ALGORTIHM AND EXPLORES:

1. Download and install Weka. You can find it here: http://www.cs.waikato.ac.nz/mn/weka/downloading.html

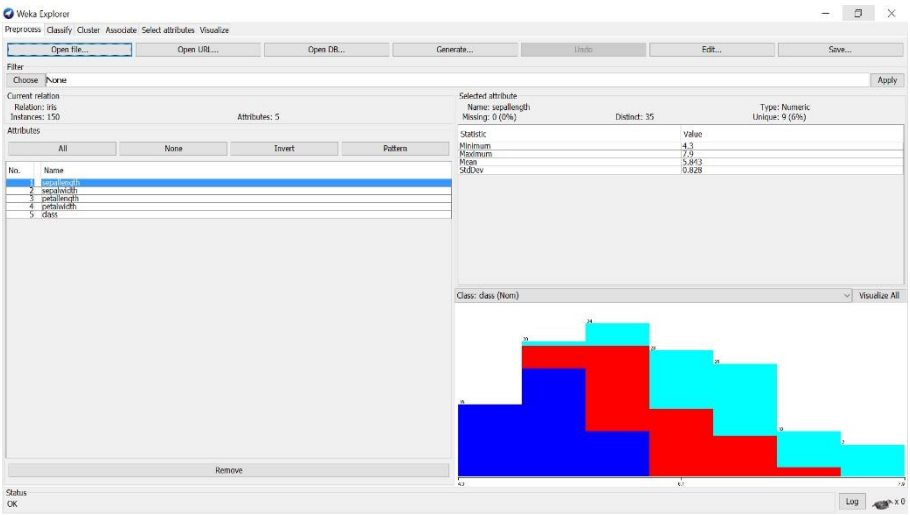
2.Open the weka tool and select the explorer option.

3.New window will be opened which consists of different options (Preprocess, Association etc.)

3. In the preprocess, click the ―open file‖ option.

4.Go to C:\Program Files\Weka-3-6\data for finding different existing. arff datasets.

Click on any dataset for loading the data then the data will be displayed as shown below



Load each dataset and observe the following:

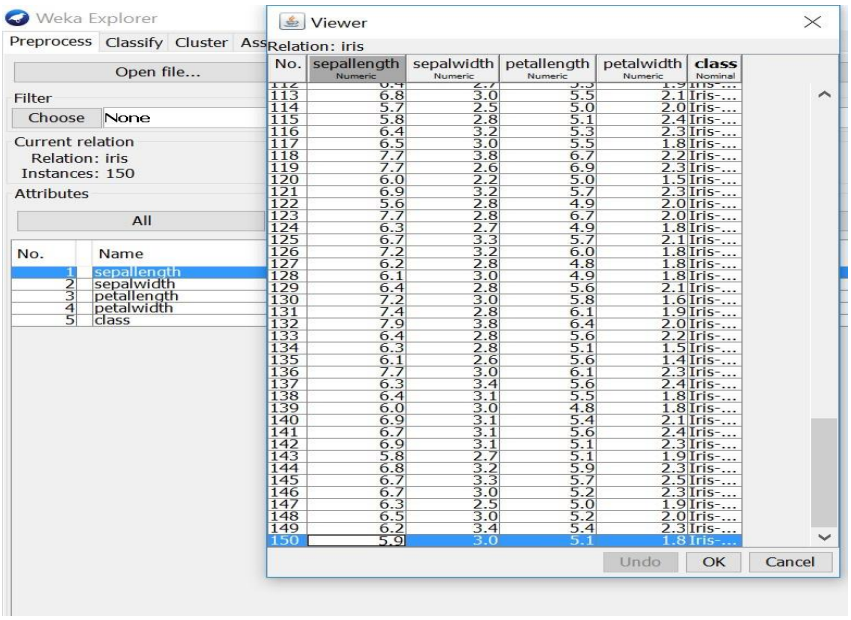
Here we have taken **IRIS.arff** dataset as sample for observing all the below things.

i. List the attribute names and they types

There are 5 attributes& its datatype present in the above loaded dataset (IRIS.arff) sepallength – Numeric sepalwidth – Numeric petallength – Numeric petallength – Numeric Class – Nominal

ii. Number of records in each dataset

There are total <u>150</u> records (Instances) in dataset (IRIS.arff).

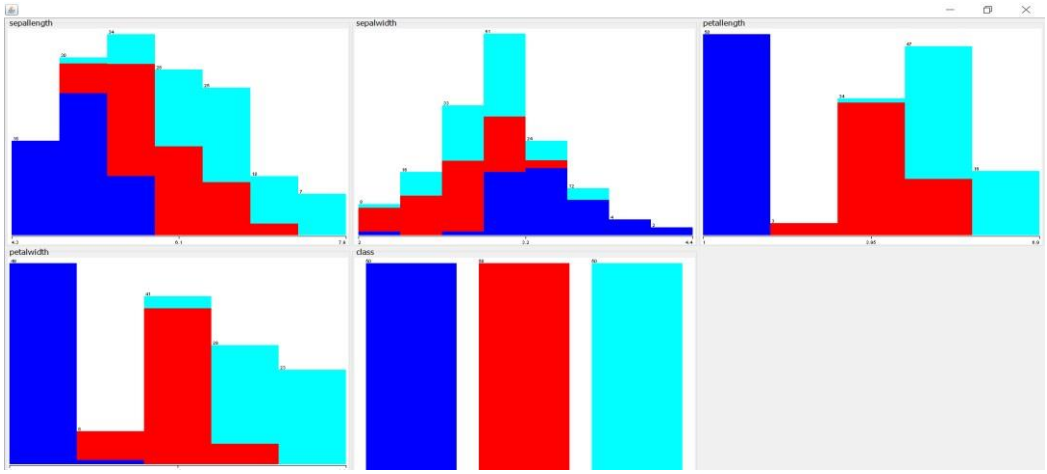


iii. Identify the class attribute (if any)

There is one class attribute which consists of 3 labels. They are: 1. Iris-setosa

2. Iris-versicolor
3. Iris-virginica

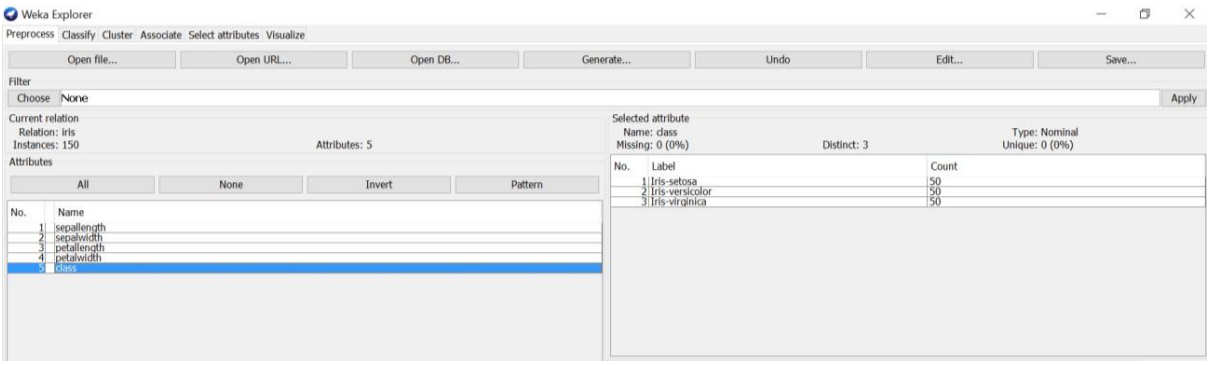
iv. Plot Histogram



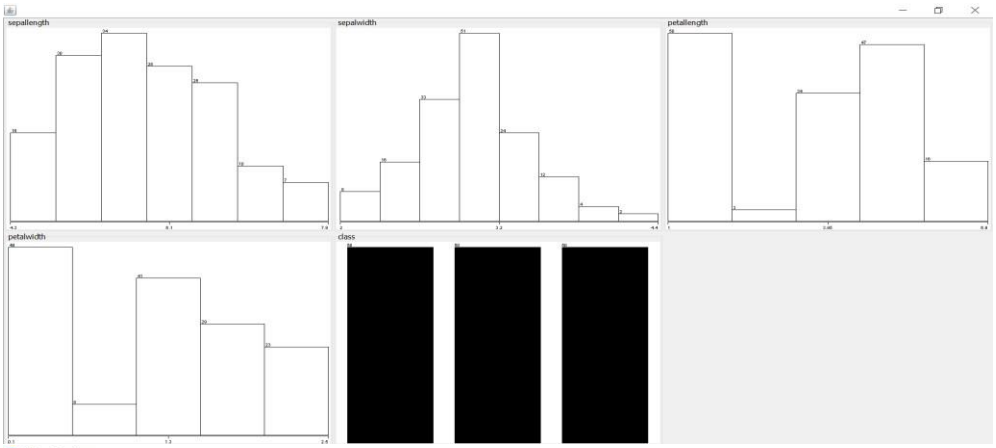
v. Determine the number of records for each class.

There is one class attribute (150 records) which consists of 3 labels. They are shown below 1. Iris-setosa - 50 records

2. Iris-versicolor – 50 records
3. Iris-virginica – 50 records



vi. Visualize the data in various dimensions





RESULT:

Thus the data exploration and integration with WEKA executed successfully.

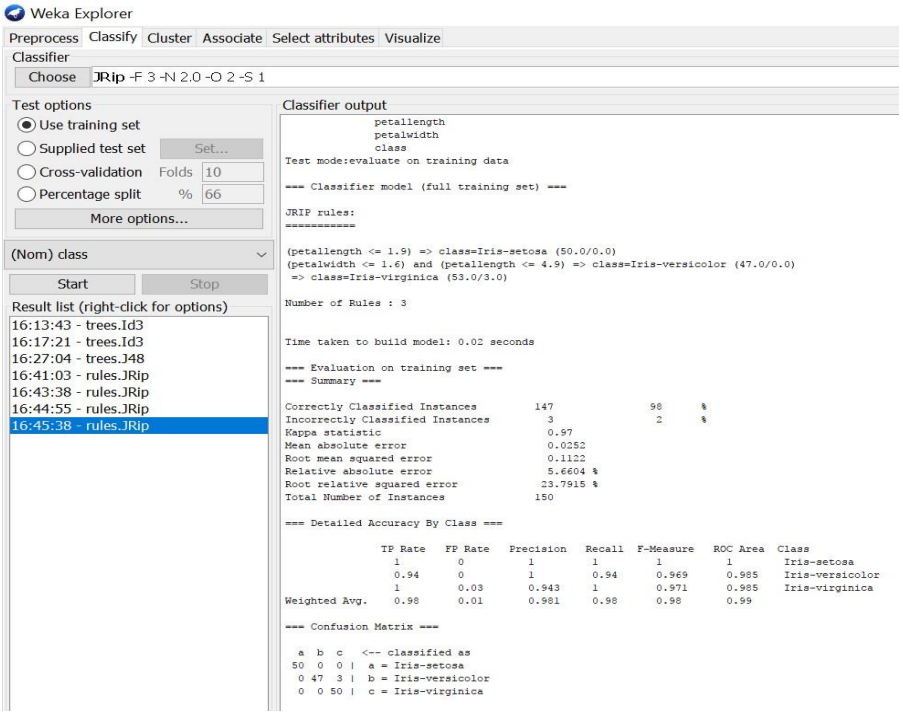
EX NO: 2	Apply WEKA tool for Data Validation
DATE:	

AIM:

To Apply WEKA tool for Data Validation

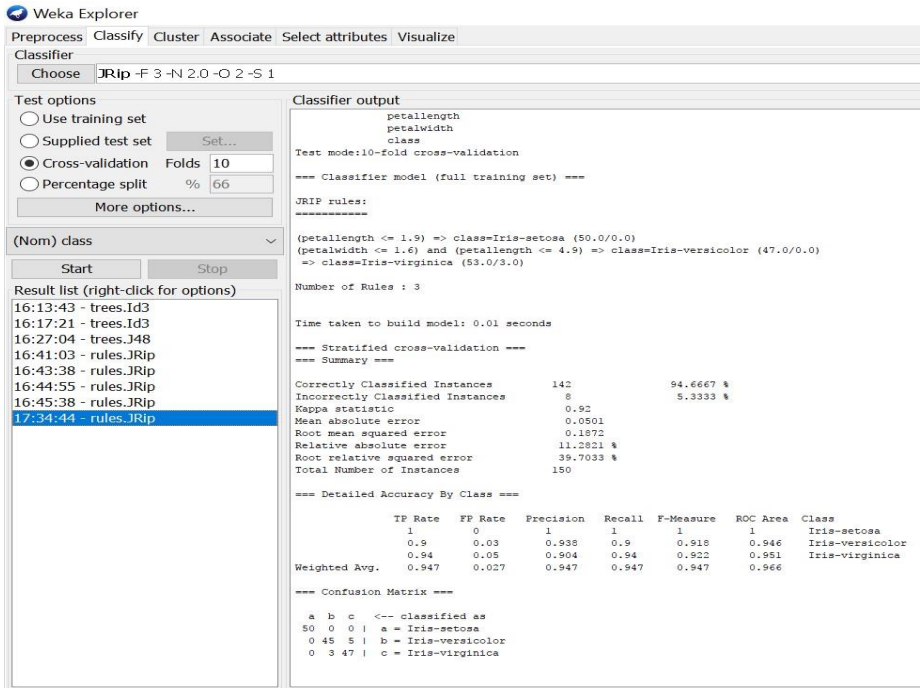
Steps and Apply:

1. Load the dataset (Iris-2D. arff) into weka tool
2. Go to classify option & in left-hand navigation bar we can see differentclassification algorithms under rules section.
3. In which we selected JRip (If-then) algorithm & click on start option with ―use training set‖ test option enabled.
4. Then we will get detailed accuracy by class consists ofF-measure, TP rate, FP rate, Precision, Recall values& Confusion Matrix as represented below.



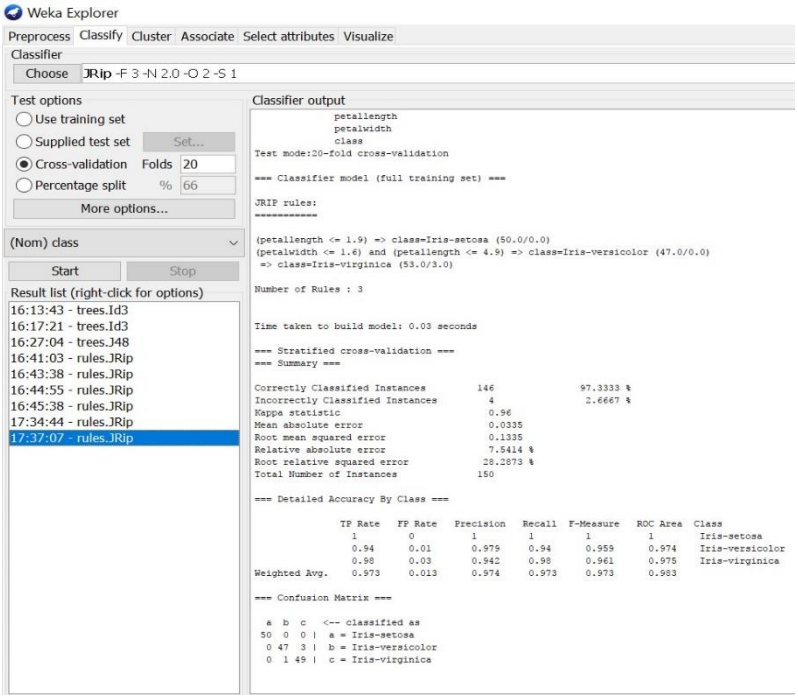
Using Cross-Validation Strategy with 10 folds:

Here, we enabled cross-validation test option with 10 folds & clicked start button as represented below.



Using Cross-Validation Strategy with 20 folds:

Here, we enabled cross-validation test option with 20 folds & clicked start button as represented below.



If we see the above results of cross validation with 10 folds & 20 folds. As per our observation the error rate is lesser with 20 folds got 97.3% correctness when compared to 10 folds got 94.6% correctness.

RESULT: Thus the WEKA tool for Data Validation done Successfully.

EXNO: 3	Plan the architecture for real time application
DATE:	

Aim:

To plan the architecture for a real-time application using Weka, you need to consider several factors. Weka is a popular machine learning library that provides various algorithms for data mining and predictive modelling.

Here are the steps to plan the architecture:

1. **Define the problem**: Clearly understand the problem you are trying to solve with your real-time application. Identify the specific tasks and goals you want to achieve using Weka.
2. **Data collection and preprocessing**: Determine the data sources and collect the required data for your application. Preprocess the data to clean, transform, and prepare it for analysis using Weka. This may involve tasks like data cleaning, feature selection, normalization, and handling missing values.
3. **Choose the appropriate Weka algorithms**: Weka offers a wide range of machine learning algorithms. Select the algorithms that are suitable for your problem and data. Consider factors like the type of data (classification, regression, clustering), the size of the dataset, and the computational requirements.
4. **Real-time data streaming**: If your application requires real-time data processing, you need to set up a mechanism to stream the data continuously. This can be done using technologies like Apache Kafka, Apache Flink, or Apache Storm. Ensure that the data streaming infrastructure is integrated with Weka for seamless processing.
5. **Model training and evaluation**: Train the selected Weka algorithms on your training dataset. Evaluate the performance of the models using appropriate evaluation metrics like accuracy, precision, recall, or F1-score. Fine-tune the models if necessary.
6. **Integration and deployment**: Integrate the trained models into your real-time application. This may involve developing APIs or microservices to expose the models' functionality. Ensure that the application can handle real-time requests and provide predictions or insights in a timely manner.
7. **Monitoring and maintenance**: Set up monitoring mechanisms to track the performance of your real-time application. Monitor the accuracy and performance of the models over time. Update the models periodically to adapt to changing data patterns or to improve performance.

Remember to document your architecture design and implementation decisions for future reference. Regularly review and update your architecture as your application evolves and new requirements arise.

RESULT: Thus architecture for real time applications was Planned.

EXNO: 4	. Write the query for schema definition
DATE:	

AIM:

To Write the query for schema definition

ALGORITHM:

1. Create a new database
2. Switch to the newly created database
3. Define the schema for each table
4. Define relationships between tables (if needed)
5. Execute the schema definition queries

PROGRAM:

```
-- Create a new database named "library"
CREATE DATABASE library;

-- Switch to the "library" database
USE library;

-- Define the schema for the "books" table
CREATE TABLE books (
book_id INT AUTO_INCREMENT PRIMARY KEY,
   title VARCHAR(255) NOT NULL,
   author VARCHAR(100) NOT NULL,
publication_year INT,
isbnVARCHAR(20),
   available BOOLEAN DEFAULT TRUE
);

-- Define the schema for the "members" table
CREATE TABLE members (
member_id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   email VARCHAR(255) UNIQUE,
phone_numberVARCHAR(20),
   address VARCHAR(255)
);

-- Define the schema for the "checkouts" table
CREATE TABLE checkouts (
checkout_id INT AUTO_INCREMENT PRIMARY KEY,
book_id INT NOT NULL,
member_id INT NOT NULL,
checkout_date DATE NOT NULL,
```

```
return_date DATE,
   FOREIGN KEY (book_id) REFERENCES books(book_id),
   FOREIGN KEY (member_id) REFERENCES members(member_id)
);
```

OUTPUT:

```
Database 'library' created.

Database changed to 'library'.

Table 'books' created successfully.

Table 'members' created successfully.

Table 'checkouts' created successfully
```

RESULT:

Thus Schema Definition was written and executed Successfully.

EXNO: 5	Design Data Ware House For Real Time Applications
DATE:	

AIM:

To Design data ware house for real time applications

ALGORITHM AND PROGRAM:

1. *Data Sources and Integration*:

sql

```
-- Example: Creating a Snowpipe for real-time data ingestion from an external stage
CREATE PIPE snowpipe_real_time
AUTO_INGEST = TRUE
AS
COPY INTO temperature_data
FROM (SELECT $1::timestamp, $2::int, $3::float FROM @real_time_stage)
FILE_FORMAT = (TYPE = 'JSON');
```

2. *Data Storage and Modeling*:

sql

```
-- Example: Creating tables for storing real-time data
CREATE TABLE temperature_data (
    timestamp TIMESTAMP,
sensor_id INT,
    temperature FLOAT
);
```

3. *Data Governance and Security*:

sql

```
-- Example: Creating roles and granting privileges
CREATE ROLE analyst_role;
GRANT USAGE ON DATABASE my_database TO analyst_role;
GRANT SELECT ON temperature_data TO analyst_role;
```

4. *Monitoring and Performance Optimization*:

sql

```
-- Example: Monitoring query performance using Snowflake's query history
SELECT * FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY_BY_USER('ANALYST_USER'));
```

5. *Deployment and Testing*:

- Deployment would involve setting up Snowflake accounts, databases, and resources, which are typically done through the Snowflake web interface or via Snowflake's APIs. Testing would involve validating the data ingestion process, querying data, and ensuring proper access controls.

6. *Training and Documentation*:

- Training sessions and documentation would cover topics such as Snowflake SQL syntax, data modeling best practices, and security principles.

7. *Iterative Improvement and Maintenance*:

- This would involve ongoing monitoring of system performance, optimizing queries and data models as needed, and iterating on the data warehouse design based on user feedback and evolving business requirements.

OUTPUT:

timestamp	sensor_id	temperature
2024-02-06 10:00:00	1	25.5
2024-02-06 10:01:00	2	26.3
2024-02-06 10:02:00	1	24.8
2024-02-06 10:02:30	3	27.1
2024-02-06 10:03:00	2	26.7
2024-02-06 10:04:00	1	25.2

RESULT: Thus Data Warehouse for real time application Designed.

EX.NO:6	Analyse the dimensional Modeling
DATE:	

AIM:

To Analyse the dimensional Modeling

ALGORITHM:

1. Identify the business process
2. Identify dimensional and facts
3. Design the dimensional model
4. Define relationships
5. Optimize for query performance

PROGRAM:

1. *Sales Fact Table:*

sql

```
CREATE TABLE SalesFact (
SaleID INT PRIMARY KEY,
DateID INT,
ProductID INT,
QuantitySold INT,
AmountSoldDECIMAL(10, 2)
);
```

2. *Date Dimension:*

sql

```
CREATE TABLE DateDim (
DateID INT PRIMARY KEY,
CalendarDate DATE,
   Day INT,
   Month INT,
   Year INT
);
```

```
-- Populate Date Dimension (sample data)
INSERT INTO DateDim (DateID, CalendarDate, Day, Month, Year)
VALUES
    (1, '2024-01-01', 1, 1, 2024),
    (2, '2024-01-02', 2, 1, 2024),
    -- Add more dates as needed
    ;
```

3. *Product Dimension:*

```
sql
CREATE TABLE ProductDim (
ProductID INT PRIMARY KEY,
    ProductName VARCHAR(255),
    Category VARCHAR(50),
    -- Additional attributes as needed
);

-- Populate Product Dimension (sample data)
INSERT INTO ProductDim (ProductID, ProductName, Category)
VALUES
    (101, 'Product A', 'Electronics'),
    (102, 'Product B', 'Clothing'),
    -- Add more products as needed
    ;
```

4. *Query to retrieve sales with date and product details:*

```
sql
SELECT
s.SaleID,
d.CalendarDate,
p.ProductName,
```

```
s.QuantitySold,
s.AmountSold
FROM
SalesFact s
JOIN DateDim d ON s.DateID = d.DateID
JOIN ProductDim p ON s.ProductID = p.ProductID;
```

This query retrieves sales information along with corresponding date and product details, leveraging the dimensional model.

OUTPUT:

SaleID	CalendarDate	ProductName	QuantitySold	AmountSold
1	2024-01-01	Product A	10	100.00
2	2024-01-02	Product B	5	50.00
3	2024-01-02	Product A	8	80.00

RESULT:

Thus the dimensional modelling Analysed Successfully.

EX.NO:7	Case study using OLAP
DATE:	

AIM:

To study case using OLAP

Introduction:

In this case study, we will explore how Online Analytical Processing (OLAP) technology was implemented in a retail data warehousing environment to improve data analysis capabilities and support decision-making processes. The case study will focus on a fictional retail company, XYZ Retail, and the challenges they faced in managing and analyzing their vast amounts of transactional data.

Background:

XYZ Retail is a large chain of stores with locations across the country. The company has been experiencing rapid growth in recent years, leading to an increase in the volume of data generated from sales transactions, inventory management, customer interactions, and other operational activities. The existing data management system was struggling to keep up with the demand for timely and accurate data analysis, hindering the company's ability to make informed business decisions.

Challenges:

1. Lack of real-time data analysis: The existing data warehouse system was unable to provide real-time insights into sales trends, inventory levels, and customer preferences.

2. Limited scalability: The data warehouse infrastructure was reaching its limits in terms of storage capacity and processing power, making it difficult to handle the growing volume of data.

3. Complex data relationships: The data stored in the warehouse was highly normalized, making it challenging to perform complex queries and analyze data across multiple dimensions.

Solution:

To address these challenges, XYZ Retail decided to implement an OLAP solution as part of their data warehousing strategy. OLAP technology allows for multidimensional analysis of data, enabling users to easily slice and dice information across various dimensions such as time, product categories, geographic regions, and customer segments.

Implementation:

1. Data modeling: The data warehouse was redesigned using a star schema model, which simplifies data relationships and facilitates OLAP cube creation.

2. OLAP cube creation: OLAP cubes were created to store pre-aggregated data for faster query performance. The cubes were designed to support various dimensions and measures relevant to the retail business.

3. Reporting and analysis: Business users were trained on how to use OLAP tools to create ad-hoc reports, perform trend analysis, and drill down into detailed data.

Results:

1. Improved data analysis: With OLAP technology in place, XYZ Retail was able to perform complex analyses on sales data, identify trends, and make informed decisions based on real-time insights.

2. Faster query performance: OLAP cubes enabled faster query performance compared to traditional relational databases, allowing users to retrieve data more efficiently.

3. Enhanced decision-making: The ability to analyze data across multiple dimensions helped XYZ Retail gain a deeper understanding of their business operations and customer behavior, leading to more strategic decision-making.

Conclusion:

By leveraging OLAP technology in their data warehousing environment, XYZ Retail was able to overcome the challenges of managing and analyzing vast amounts of data. The implementation of OLAP not only improved data analysis capabilities but also empowered business users to make informed decisions based on real-time insights. This case study demonstrates the value of OLAP in enhancing data analysis and decision-making processes in a retail environment.

RESULT:

Thus case study using OLAP done successfully.

EXNO:8	Case study using OTLP
DATE:	

AIM:

To study case using OTLP

Introduction:

This case study explores the implementation of the Operational Data Layer Pattern (OTLP) in a data warehousing environment to improve data integration, processing, and analytics capabilities. The case study focuses on a fictional company, Tech Solutions Inc., and how they leveraged OTLP to enhance their data warehousing operations.

Background:

Tech Solutions Inc. is a technology consulting firm that provides IT solutions to various clients. The company collects a vast amount of data from different sources, including customer interactions, sales transactions, and operational activities. The existing data warehouse infrastructure was struggling to handle the growing volume of data and provide real-time insights for decision-making.

Challenges:

1. Data silos: Data from different sources were stored in separate silos, making it difficult to integrate and analyze data effectively.

2. Real-time data processing: The existing data warehouse was not capable of processing real-time data streams, leading to delays in data analysis and decision-making.

3. Scalability: The data warehouse infrastructure was reaching its limits in terms of storage capacity and processing power, hindering the company's ability to scale with the growing data volume.

Solution:

To address these challenges, Tech Solutions Inc. decided to implement the OTLP pattern in their data warehousing environment. OTLP combines elements of both Operational Data Store (ODS) and Traditional Data Warehouse (TDW) architectures to enable real-time data processing, data integration, and analytical capabilities.

Implementation:

1. Data integration: Tech Solutions Inc. integrated data from various sources into the operational data layer, where data transformations and cleansing processes were applied.

2. Real-time processing: The OTLP architecture allowed for real-time data processing, enabling the company to analyze streaming data and generate insights in near real-time.

3. Analytics and reporting: Business users were provided with self-service analytics tools to create ad-hoc reports, perform trend analysis, and gain actionable insights from the integrated data.

Results:

1. Improved data integration: The OTLP architecture facilitated seamless integration of data from multiple sources, breaking down data silos and enabling a unified view of the company's operations.

2. Real-time analytics: With OTLP in place, Tech Solutions Inc. was able to analyze streaming data in real-time, allowing for faster decision-making and response to market trends.

3. Scalability: The OTLP architecture provided scalability to handle the growing volume of data, ensuring that the company's data warehousing operations could support future growth.

Conclusion:

By implementing the Operational Data Layer Pattern (OTLP) in their data warehousing environment, Tech Solutions Inc. was able to overcome the challenges of data silos, real-time data processing, and scalability. The adoption of OTLP not only improved data integration and analytics capabilities but also empowered business users to make informed decisions based on real-time insights. This case study highlights the benefits of leveraging OTLP in enhancing data warehousing operations for improved business outcomes.

RESULT:

Thus case study using OTLP done successfully.

EX.NO:9	**Implementation of warehouse testing.**

DATE:	

AIM:

To implement warehouse testing

Steps with program:

1. Install necessary libraries:

pip install pytest pandas

2. Create a Python script for data transformation and loading:

```
# data_transformation.py
import pandas as pd
def transform_data(input_data):
# Perform data transformation logic here
transformed_data = input_data.apply(lambda x: x * 2)
return transformed_data

def load_data(transformed_data):
 # Load transformed data into the operational data layer
transformed_data.to_csv('transformed_data.csv', index=False)
```

3. Create test cases using pytest:

```
# test_data_integration.py
import pandas as pd
import data_transformation

def test_transform_data():
input_data = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
expected_output = pd.DataFrame({'A': [2, 4, 6], 'B': [8, 10, 12]})
transformed_data = data_transformation.transform_data(input_data)
    assert transformed_data.equals(expected_output)
```

```
def test_load_data():
input_data = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
data_transformation.load_data(input_data)
loaded_data = pd.read_csv('transformed_data.csv')
    assert input_data.equals(loaded_data)
```

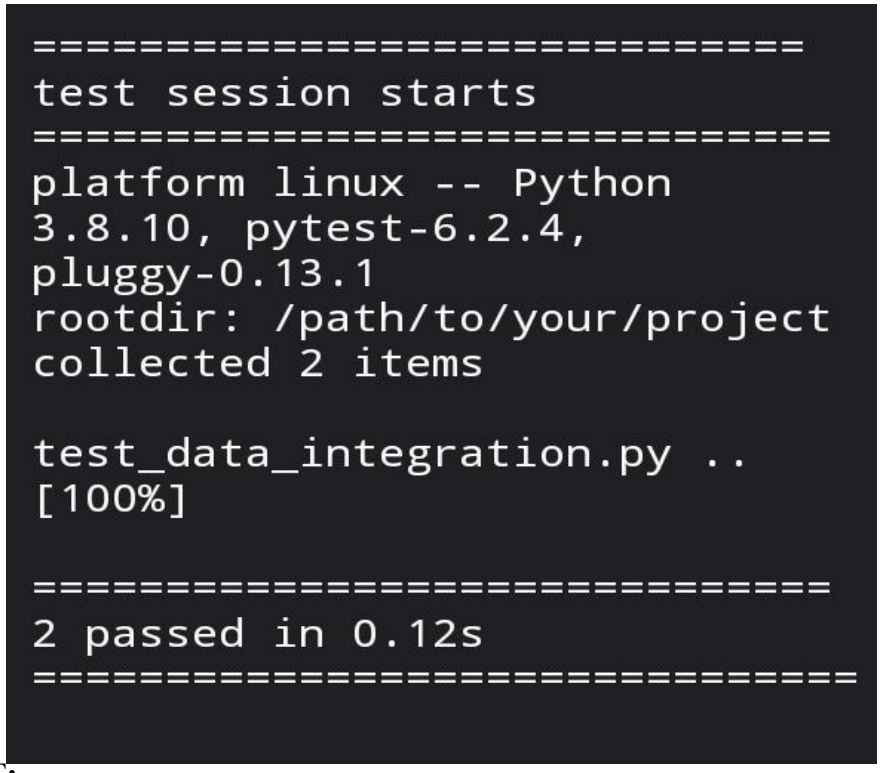
4. Run the tests using pytest:

pytest test_data_integration.py

5. Analyze the test results to ensure that the data transformation and loading processes are functioning correctly in the operational data layer.

By implementing automated tests for data integration processes in the data warehousing environment, you can ensure the accuracy and reliability of the data transformation and loading operations. This approach helps in identifying any issues or discrepancies early on in the development cycle, leading to a more robust and efficient data warehousing system.

OUTPUT:



RESULT:

Thus implementation of warehouse testing done successfully.